LINUX INTERVIEW PREPARATION GUIDE

CHAPTER

16

Real-World Linux Production Scenarios and Troubleshooting

Interview-focused notes for DevOps, SysAdmin and Cloud Engineers (1–5 years of experience)

IN THIS CHAPTER

01 Introduction
02 Troubleshooting Approach
03 SSH Connection Failing
04 Website Not Accessible
05 Disk Space Full
06 Service Fails to Start
07 Permission Denied in Deployment
08 DNS Resolution Failing
09 SSL/TLS Certificate Problem
10 High CPU Usage
11 Application Killed by OOM
12 Nginx 502 Bad Gateway
13 Deployment OK, App Broken
14 Advanced Tools
15 Checklist and Best Practices
16 Interview Q&A
17 Takeaways and Exercises

Ravi Tomar

Linux · DevOps · Cloud Infrastructure

CHAPTER OBJECTIVE

Apply everything from earlier chapters to real production incidents. Learn a structured troubleshooting approach and work through the most common scenarios: SSH failures, unreachable websites, full disks, failing services, permission errors, DNS and TLS problems, CPU and memory pressure, 502 errors and broken deployments. Each scenario points to the chapter where the tools are covered in depth.

1 Introduction

Knowing commands is not enough. A DevOps engineer must investigate and resolve real incidents. Production problems can occur at any layer:

- Operating system, CPU, memory and storage.
- Network and DNS.
- Application and web server.
- Permissions and security.
- SSL/TLS certificates.
- Dependencies, deployment and configuration.

The goal is not just to make the error disappear. It is to find the actual cause, restore service safely and prevent the problem from happening again.

2 A Structured Troubleshooting Approach

Step 1 Understand the problem

- What exactly is failing, and for whom: one user or everyone?
- Is the whole server down, or only one application?
- When did it start? Was there a recent deployment or configuration change?

Step 2 Check the current system state

```
uptime        # load and how long since reboot
free -h       # memory and swap
df -h; df -i  # disk space and inodes
top           # busiest processes
```

Step 3 **Identify the failing layer**

For an unavailable website, check in order: server reachable → application process running → listening on the right port → firewall and security group → web server → DNS → TLS → application response.

Step 4 **Collect evidence**

Check logs, service status, processes, connections and configuration. Avoid restarting services or killing processes before you understand the problem; evidence can disappear.

Step 5 **Apply a safe fix**

For example: correct the configuration, restore permissions, free disk space safely, restart after validating configuration, or roll back the deployment.

Step 6 **Verify the fix**

```
systemctl status nginx
ss -tulnp
curl -I http://localhost
journalctl -u nginx -n 50
```

Verify from the server and from an external client where possible.

Step 7 **Find and document the root cause**

Record what happened, why it happened, how it was fixed and what will prevent it next time.

3 Scenario 1: SSH Connection Is Failing

```
ssh ubuntu@10.0.1.25
```

The error message tells you which layer to investigate:

Error	Likely meaning
Connection timed out	Packets are dropped: security group, NACL, routing or firewall
Connection refused	Server is reachable, but nothing listens on port 22 (or a firewall rejects it)
Permission denied (publickey)	Wrong username, wrong key, or bad permissions on the server's `~/.ssh`
No route to host	Routing or network connectivity problem

```
# From your machine: verbose output shows where it fails
ssh -v -i my-key.pem ubuntu@10.0.1.25

# Private key must not be readable by others
chmod 400 my-key.pem

# On the server (via console or Session Manager)
sudo systemctl status ssh          # sshd on RHEL / Amazon Linux
sudo ss -tlnp | grep ':22'
sudo journalctl -u ssh -n 50       # or -u sshd
sudo tail -n 50 /var/log/auth.log  # /var/log/secure on RHEL
```

Default usernames: `ubuntu` (Ubuntu), `ec2-user` (Amazon Linux, RHEL), `admin` (Debian).

On EC2, also check the security group (port 22 from your IP), NACL, route table and public IP, using the checklist in Chapter 7.

> **IMPORTANT**
>
> Do not open SSH to `0.0.0.0/0` permanently to "fix" access. Restrict it to trusted IPs, or use AWS Systems Manager Session Manager, which needs no inbound port.

4 Scenario 2: Website Is Not Accessible

A user reports that `https://example.com` is not opening. Work through the layers.

```
# 1. Is the server reachable on the web ports? (ping may be blocked)
nc -zv SERVER_IP 80
nc -zv SERVER_IP 443

# 2. Are the web server and application running?
sudo systemctl status nginx
ps aux | grep -E '[n]ode|[p]hp-fpm'

# 3. What is listening, and on which address?
sudo ss -tlnp
```

```
LISTEN  0  511    0.0.0.0:80      users:(("nginx",...))
LISTEN  0  511    0.0.0.0:443     users:(("nginx",...))
LISTEN  0  511  127.0.0.1:3000    users:(("node",...))
```

Here Nginx is public on 80 and 443, and the Node.js app listens only on localhost port 3000. That is the normal reverse-proxy design.

Test Each Layer from the Inside Out

```
curl -I http://127.0.0.1:3000   # the application directly
curl -I http://localhost        # through Nginx
curl -I http://SERVER_IP        # through the server's real IP
```

The first layer that fails is where the problem is. If it works locally but not from outside, check the firewall, security group and DNS.

When the App Must Be Reached Directly

If `ss` shows `127.0.0.1:3000`, other machines cannot connect to it. Either bind the app to `0.0.0.0:3000` or, better, keep it on localhost and publish it through an Nginx reverse proxy:

```
server {
    listen 80;
    server_name example.com;

    location / {
        proxy_pass http://127.0.0.1:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

```
sudo nginx -t && sudo systemctl reload nginx
sudo tail -n 50 /var/log/nginx/error.log
```

Common causes: application stopped, invalid Nginx configuration, wrong port or bind address, firewall or security group blocking traffic, wrong proxy target, DNS pointing to the wrong IP, expired certificate.

5 Scenario 3: Disk Space Is Full

The application stops writing logs or accepting uploads with `No space left on device`. Full details are in Chapter 6.

```
# 1. Which filesystem is full?
df -h

# 2. Which directories are largest?
sudo du -h --max-depth=1 /var | sort -hr | head

# 3. Which files are largest?
sudo find /var -xdev -type f -size +500M -ls

# 4. Are inodes exhausted instead of space?
df -i

# 5. Are deleted files still held open by a process?
sudo lsof +L1
```

Safe fixes: rotate or compress old logs, truncate (not delete) a huge active log, vacuum the journal, clear package caches, remove confirmed temporary files, expand the volume, fix excessive application logging and configure logrotate (see Chapter 12).

> **WARNING**
>
> Never blindly delete files from `/var`, `/var/lib` or application directories. Deleting a log that a process still has open frees nothing until the process restarts or reopens its logs.

6 Scenario 4: A Service Fails to Start

```
Job for myapp.service failed because the control process exited with error code.
```

```
sudo systemctl status myapp
sudo journalctl -u myapp -n 100
sudo journalctl -u myapp --since "15 minutes ago"

# The unit file systemd is actually using
systemctl cat myapp
```

Check the unit file (see Chapter 5):

```
[Service]
User=appuser
WorkingDirectory=/var/www/myapp
ExecStart=/usr/bin/node /var/www/myapp/server.js
Restart=on-failure
```

- Does `appuser` exist, and can it read the application files?
- Does the working directory exist?
- Is the executable path correct (`which node`)?
- Are required environment variables provided (`Environment=` or `EnvironmentFile=`)?
- Is the port already in use by another process?

```
# After editing the unit
sudo systemctl daemon-reload
sudo systemctl restart myapp
sudo systemctl status myapp
```

INTERVIEW TRAP

"It works when I run it manually" does not mean it works under systemd. The service runs as a different user, with a minimal environment, a different working directory and no shell profile.

7 Scenario 5: Permission Denied During Deployment

```
cp: cannot create regular file '/var/www/html/index.php': Permission denied
```

```
# Who is the pipeline running as?
whoami
id

# Who owns the target directory?
ls -ld /var/www /var/www/html

# Permissions of every directory in the path
namei -l /var/www/html/index.php
```

For example, the CI/CD agent runs as `gitlab-runner`, but the directory belongs to `www-data`. A group-based approach fixes this without opening permissions to everyone:

```
sudo usermod -aG www-data gitlab-runner
sudo chown -R deploy:www-data /var/www/myapp
sudo chmod 2775 /var/www/myapp          # SGID: new files keep the www-data group
sudo systemctl restart gitlab-runner     # new group membership needs a new session
```

Choose ownership to fit your deployment design: the deploy user writes the code, the runtime user only reads it. See Chapter 3 for SGID and ACLs.

> **WARNING**
> Never use `chmod -R 777` as a fix. It lets every user and every compromised process modify your application.

8 Scenario 6: DNS Resolution Is Failing

`curl http://203.0.113.10` works, but `curl http://example.com` fails. The network is fine; name resolution is not. Full details are in Chapter 7.

```
# What the system resolver returns (includes /etc/hosts)
getent hosts example.com

# What DNS returns (ignores /etc/hosts)
dig +short example.com

# Compare with a public resolver to isolate the local resolver
dig @8.8.8.8 +short example.com

# Which resolver is configured?
cat /etc/resolv.conf
resolvectl status          # Ubuntu (systemd-resolved)

# Any local override?
grep example.com /etc/hosts
```

Result	Points to
Public resolver works, local does not	Local resolver configuration or the DNS server it uses
Both return the wrong IP	Wrong DNS record, or caching (TTL) after a recent change
`getent` differs from `dig`	An entry in `/etc/hosts`
Both time out	Outbound DNS (UDP/TCP 53) blocked by a firewall or security rule

> **PRODUCTION TIP**
>
> If resolution fails on only one server, compare its `/etc/resolv.conf`, `/etc/hosts` and network rules with a working server.

9 Scenario 7: SSL/TLS Certificate Problem

Users see `NET::ERR_CERT_DATE_INVALID`, or clients report TLS handshake errors.

```
# Validity dates of the certificate the server actually presents
openssl s_client -connect example.com:443 -servername example.com </dev/null 2>/dev/null
  | openssl x509 -noout -dates -subject -issuer

# Does the private key match the certificate? (the two hashes must be identical)
sudo openssl x509 -in /etc/letsencrypt/live/example.com/fullchain.pem -noout -pubkey | sh
sudo openssl pkey -in /etc/letsencrypt/live/example.com/privkey.pem -pubout | sha256sum

# Is the server clock correct?
timedatectl
```

```
server {
    listen 443 ssl;
    server_name example.com;
    ssl_certificate     /etc/letsencrypt/live/example.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/example.com/privkey.pem;
}
```

```
sudo nginx -t && sudo systemctl reload nginx

# Test Let's Encrypt renewal without changing anything
sudo certbot renew --dry-run
```

Common causes: expired certificate, wrong file path, private key does not match, missing intermediate certificate (use `fullchain.pem`, not `cert.pem`), hostname mismatch, wrong server time.

> **IMPORTANT**
>
> Automatic renewal is not enough on its own. The web server must be reloaded to load the new certificate, and expiry should be monitored so you are alerted well in advance.

10 Scenario 8: High CPU Usage

The server is slow and response times increase. Full details are in Chapter 13.

```
uptime                                      # load trend
nproc                                       # compare load with CPU count
ps aux --sort=-%cpu | head                  # top consumers
ps -p PID -o pid,ppid,%cpu,%mem,etime,cmd   # details and how long it has run
```

Remember that load average also counts tasks waiting on I/O, so confirm with CPU metrics before blaming the CPU.

Common causes: inefficient code or loops, too many background jobs, traffic spikes, expensive database queries, misconfigured worker counts.

Safe fixes: fix the expensive task, adjust workers, rate-limit abusive traffic, roll back a bad deployment, or scale when justified. Do not kill the top process blindly; it may be your main production service.

11 Scenario 9: Application Killed by Memory Pressure

The application stops or restarts unexpectedly, and its logs show no clean shutdown.

```
free -h
ps aux --sort=-%mem | head
sudo journalctl -k | grep -iE 'oom|out of memory|killed process'
```

Common causes: memory leak, too many workers, traffic increase, insufficient RAM, oversized caches, or a container memory limit that is too low (see Chapters 13 and 15).

Fixes: find and fix the leak, tune worker counts and caches, set sensible limits, and add memory when the workload genuinely needs it.

> **IMPORTANT**
>
> Adding swap is not always the answer. It can absorb short spikes, but it does not fix a memory leak, and heavy swapping makes the server very slow.

12 Scenario 10: Nginx Returns 502 Bad Gateway

A 502 means Nginx could not get a valid response from the upstream application. (A 504 means the upstream was reached but took too long; see Chapter 7.)

```
# Is the backend running and answering locally?
sudo systemctl status myapp
curl -v http://127.0.0.1:3000

# What does Nginx say?
sudo tail -n 100 /var/log/nginx/error.log
```

```
connect() failed (111: Connection refused) while connecting to upstream,
upstream: "http://127.0.0.1:3000/"
```

`Connection refused` means nothing is listening on that port: the backend is down, or the `proxy_pass` port is wrong.

```
sudo nginx -t && sudo systemctl reload nginx
```

Common causes: backend stopped or crashed, wrong upstream IP or port, backend bound to a different interface, backend overloaded, firewall between Nginx and backend.

> **INTERVIEW TRAP (RHEL)**
>
> If `curl` to the backend works but Nginx still returns 502 on a RHEL-based system, check SELinux. Web servers are not allowed to make network connections by default: `sudo setsebool -P httpd_can_network_connect on` (see Chapter 9).

13 Scenario 11: Deployment Succeeds but the Application Does Not Work

The pipeline is green, but the application is down. This is one of the most common real incidents.

```
# 1. Were the right files deployed, with the right ownership?
ls -lah /var/www/myapp

# 2. Is the service running, and what did it log at startup?
sudo systemctl status myapp
sudo journalctl -u myapp -n 100

# 3. Is it listening on the expected port?
sudo ss -tlnp

# 4. Does the backend answer, and is the proxy healthy?
curl -v http://127.0.0.1:3000
sudo nginx -t

# 5. What changed compared to the last working release?
diff -u /etc/myapp/app.env.bak /etc/myapp/app.env
```

Common root causes: wrong environment variables, missing configuration file, wrong permissions, missing dependency, wrong runtime version, app started on a different port, database unreachable, a deployment overwriting an important file.

PRODUCTION BEST PRACTICE

A pipeline should not finish green until a **post-deployment health check** passes, for example `curl -fsS http://127.0.0.1:3000/health`. Keep the previous release available for a fast rollback, such as a `current` symlink (see Chapter 2), and keep environment configuration separate from code.

14 Advanced Troubleshooting Tools

Tool	Use it to	Example
`lsof`	Find which process uses a port or file	`sudo lsof -i :3000`
`fuser`	Find the process blocking a port or mount	`sudo fuser -v 80/tcp`
`strace`	See which system call fails or hangs	`sudo strace -f -p PID`
`tcpdump`	Confirm whether packets arrive and leave	`sudo tcpdump -nn -i any port 443`
`timeout`	Stop a check from hanging forever in scripts	`timeout 10s curl -v http://localhost:3000`

Details are in Chapter 7 (`tcpdump`), Chapter 11 (`lsof`) and Chapter 14 (`strace`).

WARNING

`strace` slows the traced process, and packet captures can contain passwords, tokens and personal data. Keep both short and targeted, and protect any saved output.

15 Troubleshooting Checklist and Best Practices

Area	Commands / Checks
Server health	`uptime`, `top`, `free -h`
Disk	`df -h`, `df -i`, `du -h`, `lsof +L1`
Processes	`ps aux`, `pgrep -a`
Services	`systemctl status`, `systemctl --failed`
Logs	`journalctl`, `/var/log`
Network	`ip addr`, `ip route`, `ss -tlnp`
Connectivity	`nc -zv`, `curl -v`, `traceroute`
DNS	`getent hosts`, `dig`, `/etc/hosts`
Permissions	`id`, `ls -l`, `namei -l`, SELinux context
TLS	`openssl s_client`, `timedatectl`
Deep inspection	`lsof`, `fuser`, `strace`, `tcpdump`

1. **Understand before changing:** do not restart or kill anything until you know why.
2. **Check recent changes first:** most incidents follow a deployment or configuration change.
3. **Read the logs:** they usually contain the answer.
4. **Follow the layers:** test application, service, network, firewall and DNS separately.
5. **Validate configuration** before every reload.
6. **Protect production data:** never delete unknown files to free space.
7. **Back up** configuration and data before risky changes.
8. **Verify from the user's side** after the fix.
9. **Document the incident**: root cause, fix and prevention.

16 Interview Questions and Answers

Q1. A website is not accessible. How do you troubleshoot it?

I check whether the server is reachable on ports 80/443, whether the web server and application are running, and what is listening with `ss -tlnp`. I test each layer with `curl` from the inside out (app, then Nginx, then server IP), read the Nginx and application logs, and check the firewall, security group, DNS and certificate. The first layer that fails is where I focus.

Q2. A Linux server's disk is 100% full. What do you do?

I use `df -h` to find the full filesystem, `du` and `find` to locate large directories and files, `df -i` for inode exhaustion, and `lsof +L1` for deleted files still held open. I remove only confirmed unnecessary data, truncate rather than delete active logs, configure log rotation, and expand storage if needed.

Q3. How do you troubleshoot a service that fails to start?

I check `systemctl status` and `journalctl -u`, then verify the unit file: user, working directory, executable path, environment variables, permissions and port conflicts. After fixing it, I run `daemon-reload`, restart and verify.

Q4. What is the difference between "connection refused" and "connection timed out"?

Refused means the host is reachable but nothing accepted the connection on that port, or a firewall actively rejected it. Timed out means no reply came back at all, usually because a firewall, security group or routing problem dropped the packets.

Q5. How do you troubleshoot high CPU usage?

I compare load with `nproc`, find the top consumers with `top` or `ps aux --sort=-%cpu`, then investigate that process's logs, workload and recent deployments before acting.

Q6. What causes an Nginx 502 Bad Gateway?

Nginx cannot get a valid response from the backend: the backend is down, the `proxy_pass` port or address is wrong, the backend is overloaded, or something blocks the connection, such as SELinux on RHEL. I check the backend with `curl` and read the Nginx error log.

Q7. How do you find which process is using a port?

```
sudo ss -tlnp | grep :8080
sudo lsof -i :8080
```

Q8. How do you fix permission errors in a CI/CD deployment?

I check which user the agent runs as with `id`, inspect ownership along the path with `namei -l`, then use group membership, correct ownership and SGID directories, never `chmod 777`.

Q9. An application keeps getting killed. What do you check?

Memory usage with `free -h` and `ps aux --sort=-%mem`, and the kernel log for OOM events with `journalctl -k`. Then I look for a memory leak, excessive workers or a container limit that is too low.

Q10. How do you troubleshoot an SSL certificate problem?

I check the presented certificate's dates, hostname and issuer with `openssl s_client`, confirm the key matches the certificate, check that the full chain is configured, check the server time, then validate and reload the web server.

Q11. The pipeline succeeded but the application is down. What do you do?

I check the deployed files and ownership, service status and startup logs, the listening port, and a local `curl` to the backend, then compare configuration with the last working release. If the cause is not quick to fix, I roll back first and investigate after service is restored.

Q12. What is your general approach to any production incident?

Understand the impact and timeline, check recent changes, collect evidence from logs and metrics, isolate the failing layer, apply the safest fix (often a rollback), verify from the user's perspective, and document the root cause and prevention.

17 Chapter Takeaways

After completing this chapter, you should be able to:

- ✓ Follow a structured troubleshooting process from symptom to root cause.
- ✓ Use the SSH error message to identify the failing layer.
- ✓ Test a website layer by layer, from the application to the outside.

- ✓ Handle full disks, inode exhaustion and deleted-but-open files safely.
- ✓ Diagnose failing systemd services.
- ✓ Fix deployment permission problems without insecure permissions.
- ✓ Separate DNS problems from network problems.
- ✓ Diagnose certificate expiry, chain and key mismatch problems.
- ✓ Investigate CPU and memory pressure.
- ✓ Troubleshoot Nginx 502 errors, including SELinux on RHEL.
- ✓ Investigate broken deployments and roll back safely.
- ✓ Use `lsof`, `fuser`, `strace` and `tcpdump` when needed.

PRACTICAL EXERCISES

Use a test server or local VM. Never disrupt a production server on purpose.

1. **Website:** install Nginx, verify port 80 with `ss` and `curl`, stop Nginx, observe the error, then restore it.
2. **Disk:** create a large test file, observe `df -h` and `du`, keep it open with `tail -f`, delete it, and find it with `lsof +L1`.
3. **Service:** create a systemd service with a wrong `ExecStart` path, read the error in `journalctl`, then fix it.
4. **Permissions:** create a deployment user and a restricted directory, reproduce "Permission denied", then fix it with a group and SGID.
5. **Network:** run a test web server on port 8080, test it with `curl`, stop it, and compare the "refused" error with a firewall-dropped "timed out" error.
6. **Reverse proxy:** proxy Nginx to a Node.js app on port 3000, stop the app, read the 502 in the Nginx error log, then restore it.
7. **TLS:** check the expiry date and issuer of a public website's certificate with `openssl`.

KEY TAKEAWAY

A good DevOps engineer does not restart services until the problem disappears. They collect evidence, isolate the root cause, apply a safe fix and verify that the system is healthy.

NEXT CHAPTER

Chapter 17: Linux in AWS and Cloud Environments

You will learn how to operate Linux servers on AWS, including EC2 user data and cloud-init, IAM roles, Systems Manager and cloud-specific troubleshooting.

END OF CHAPTER 16